

NEW L.J. INSTITUTE OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Engineering / AIML / Data Science

Subject: Data Structures

Code: BE03000081 (Sem-3)

Credits: 4 (L:3, T:0, P:2)

Coverage: Units 1 to 5 (Full Solutions)

UNIT 1: INTRODUCTION TO DATA STRUCTURES

CO-1 • WEIGHTAGE: 10%

Q1. Define Data Structure. Explain its importance, classification, and applications.

[3, 4 & 7 Marks • CO1]

Definition: A data structure is a specialized programmatic format for organizing, storing, processing, and retrieving data efficiently. It establishes logical, mathematical, or physical relationships among individual data items.

Importance: (1) *Efficiency*: Minimizes algorithmic time complexity and memory overhead; (2) *Reusability*: Promotes modular software libraries; (3) *Abstraction*: Decouples user-facing interfaces from physical memory architectures.

Real-World Applications: Operating system process scheduling (Queues), expression evaluation & memory call stacks (Stacks), database indexing (B-Trees & Hash Tables), map navigation & routing (Graphs), document object models (Trees).

Q2. Differentiate between Primitive and Non-Primitive Data Structures.

[3, 4 & 7 Marks • CO1]

Attribute	Primitive Data Structures	Non-Primitive Data Structures
Definition	Basic, machine-level data types predefined in programming languages.	Complex structures derived by grouping primitive or non-primitive elements.
Memory Allocation	Fixed compile-time size allocated directly on the Stack memory.	Dynamic, variable memory typically allocated in the Heap via pointers.
Operations	Direct machine-level operations (+, -, *, /, bitwise, relational).	Algorithmic operations: traversal, searching, sorting, insertion, deletion.
Examples	int, float, char, boolean, pointer.	Arrays, Linked Lists, Stacks, Queues, Trees, Graphs.

Q3. Differentiate between Linear and Non-Linear Data Structures.

[3 & 4 Marks • CO1]

• **Linear Data Structures:** Data elements are organized in a sequential, 1-to-1 chain. Every element has a unique predecessor and successor (except endpoints). Traversal processes elements in a single sequential pass. *Examples:* Arrays, Stacks, Queues, Linked Lists.

• **Non-Linear Data Structures:** Elements are organized in hierarchical (1-to-many) or interconnected network (many-to-many) relationships. Traversal is non-sequential and multi-directional. *Examples:* Trees (hierarchical parent-child), Graphs (arbitrary network cycles).

Q4. Differentiate between Data Types and Data Structures.

[3 Marks • CO1]

• **Data Type:** An abstract language declaration that defines the domain of values a variable can hold and the allowable machine operations (e.g., int holds 4-byte signed integers). It carries no algorithmic time complexity.

• **Data Structure:** A concrete software implementation that stores, organizes, and links multiple data items together through specified access rules and algorithms (e.g., BST, Queue) with measurable time and space complexity.

Q5. What is an Abstract Data Type (ADT)? Explain its Features and Examples.

[3 & 4 Marks • CO1]

Definition: An ADT is a mathematical and logical specification of a data object along with the operations that can be performed on it, completely independent of any programming language or physical memory implementation.

Key Features: (1) *Data Abstraction*: Hides internal structures; (2) *Encapsulation*: Bundles operations into a unified interface; (3) *Implementation Independence*: A Queue ADT behaves identically whether implemented via arrays or linked lists.

Q6. Explain the Storage Representation of Primitive and Non-Primitive Data Structures.

[4 & 7 Marks • CO1]

1. Primitive Storage: Integers are stored in fixed-width words (32-bit). Positive integers use pure binary with MSB = 0. Negative integers use **2's complement** (1's complement + 1; e.g., +10 = 0000...1010 → 1's comp = 1111...0101 → +1 = 1111...0110 for -10). Floats use **IEEE 754** standard (1 sign bit, 8 exponent bits, 23 mantissa bits). Chars use 1-byte ASCII codes ('A'=65 = 01000001). Booleans use 1 bit (0/1), byte-aligned.

2. Non-Primitive Storage: Arrays are stored in contiguous memory blocks computed via $Base + Index \times Size$. Dynamic structures (Linked Lists, Trees, Graphs) are allocated in Heap memory as discrete nodes containing data fields and pointer addresses (4 or 8 bytes each).

Q7. Describe Basic Common Operations Performed on Data Structures.

[3 Marks • CO1]

(1) **Traversing:** Accessing every record exactly once; (2) **Searching:** Finding the address of a target key; (3) **Inserting:** Adding new records; (4) **Deleting:** Removing an existing record; (5) **Sorting:** Arranging items in ascending/descending order; (6) **Merging:** Combining two sorted files into a single ordered structure.

Q8. Explain Row-Major and Column-Major 2D Array Address Calculation Formulas.**[4 & 7 Marks • CO2]**

In memory, 2D arrays are flattened into sequential linear addresses using two conventions:

- **Row-Major Order (RMO):** Elements are placed row by row. Formula for array $A[L_r:U_r, L_c:U_c]$ where $M = U_r - L_r + 1$, $N = U_c - L_c + 1$, base address B , and element size W :

$$\text{Address}(A[I][J]) = B + W \times [N \times (I - L_r) + (J - L_c)]$$

- **Column-Major Order (CMO):** Elements are placed column by column. Formula:

$$\text{Address}(A[I][J]) = B + W \times [(I - L_r) + M \times (J - L_c)]$$

Q9. Solved University Numerical Problems on 2D Array Address Calculation.**[4 & 7 Marks • CO2]**

- **Problem 1:** `int a[3][4]`, Base = 1050, $W = 2$ bytes. Find address of `a[2][2]` ($L_r=0, L_c=0, M=3, N=4$):

- Row-Major: $1050 + 2 \times [4 \times (2-0) + (2-0)] = 1050 + 2 \times 10 = \mathbf{1070}$.

- Column-Major: $1050 + 2 \times [(2-0) + 3 \times (2-0)] = 1050 + 2 \times 8 = \mathbf{1066}$.

- **Problem 2:** 2D Array `A[1:4, 5:8]`, Base = 2000, $W = 2$ bytes. Find address of `A[3,7]` ($M = 4, N = 4, L_r = 1, L_c = 5$):

- Row-Major: $2000 + 2 \times [4 \times (3-1) + (7-5)] = 2000 + 2 \times [8 + 2] = \mathbf{2020}$.

- Column-Major: $2000 + 2 \times [(3-1) + 4 \times (7-5)] = 2000 + 2 \times [2 + 8] = \mathbf{2020}$.

- **Problem 3:** Array `Z_1[2:9, 9:18]`, Base = 100, $W = 4$ bytes. Find address of `Z_1[4,12]` ($M = 8, N = 10, L_r = 2, L_c = 9$):

- Row-Major: $100 + 4 \times [10 \times (4-2) + (12-9)] = 100 + 4 \times [20 + 3] = 100 + 92 = \mathbf{192}$.

- Column-Major: $100 + 4 \times [(4-2) + 8 \times (12-9)] = 100 + 4 \times [2 + 24] = 100 + 104 = \mathbf{204}$.

Q10. What is a Sparse Matrix? Explain 3-Tuple Representation and Memory Savings.**[3, 4 & 7 Marks • CO2]**

Definition: A sparse matrix is a matrix where the vast majority of elements are zero. Storing it as a standard 2D array wastes memory and processing time. It is compactly stored using a **3-Tuple (Triplet)** table of dimension $(k+1) \times 3$ where k is the number of non-zero elements.

- Header Row (Row 0): [Total Rows, Total Columns, Total Non-Zero (k)].
- Rows 1 to k : [Row Index, Column Index, Value].

Memory Calculation: Standard = $m \times n \times s$ bytes. Triplet = $(k+1) \times 3 \times s$ bytes. For a 6×7 matrix with 8 non-zero elements: Standard requires 84 bytes; Triplet requires $(8+1) \times 3 \times 2 = 54$ bytes (saves 30 bytes).

Triplet for Diagonal Matrix $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$: Header: [3, 3, 3]; Row 1: [0, 0, 1]; Row 2: [1, 1, 2]; Row 3: [2, 2, 3].

Q11. Define Stack. Give Complete Algorithms for PUSH, POP, PEEP, and CHANGE Operations.**[4 & 7 Marks • CO2]**

A stack is a linear data structure operating on the **Last-In, First-Out (LIFO)** principle. All operations occur at **TOP**.

```

Procedure PUSH(S, TOP, N, X): 1. [Overflow?] IF TOP >= N THEN Write('Stack Overflow'); Return 2. TOP = TOP + 1 3. S[TOP] = X; Return
Procedure POP(S, TOP): 1. [Underflow?] IF TOP == 0 THEN Write('Stack Underflow'); Return(0) 2. TOP = TOP - 1 3. Return(S[TOP + 1])
Procedure PEEP(S, TOP, I): 1. [Underflow?] IF (TOP - I + 1) <= 0 THEN Write('Stack Underflow on Peep'); Return(0) 2. Return(S[TOP - I + 1])
Procedure CHANGE(S, TOP, I, X): 1. [Underflow?] IF (TOP - I + 1) <= 0 THEN Write('Stack Underflow on Change'); Return 2. S[TOP - I + 1] = X; Return

```

Multiple Stacks: Two stacks can share a single array of size N without collision: Stack 1 starts at index 0 and grows upwards ($TOP1++$), while Stack 2 starts at $N-1$ and grows downwards ($TOP2--$). Overflow occurs only when $TOP1 + 1 == TOP2$, maximizing memory utilization.

Q12. Explain Operator Precedence & Associativity and Infix-to-Postfix Algorithm (REVPOL).**[4 & 7 Marks • CO2]**

Precedence Rules: Exponentiation ($\$^{\$}$ or $\$ \backslash \$$, right-associative, highest: $f=6, g=5$) > Multiplication/Division ($*$, $/$, left-associative: $f=3, g=4$) > Addition/Subtraction ($+$, $-$, left-associative, lowest: $f=1, g=2$). Parentheses: '(' has input precedence 9, stack precedence 0.

Algorithm REVPOL(INFIX): (1) Pad input with ')', push '(' to stack with $Top=1$. (2) Read tokens using NEXTCHAR. (3) Operands pass directly to output. (4) For operators, pop higher/equal precedence operators from stack to output, then push current operator. (5) ')' pops and appends operators until '(' is matched and discarded. Output expression is Reverse Polish Notation (RPN).

Q13. Solved Infix to Postfix Conversion Traces.**[4 & 7 Marks • CO2]**

- **Trace 1:** $A * (B + C) - D \rightarrow$ Postfix = $A \ B \ C \ + \ * \ D \ -$
- **Trace 2:** $(A - B) / C * D \wedge (E / F) \wedge (G + H) \rightarrow$ Postfix = $A \ B \ - \ C \ / \ D \ E \ F \ / \ G \ H \ + \ ^ \ ^ \ *$
- **Trace 3:** $((A - (B + C)) \wedge D) / (E + F) \rightarrow$ Postfix = $A \ B \ C \ + \ - \ D \ * \ E \ F \ + \ /$
- **Trace 4:** $((A - B) + D / ((E + F) * G)) \rightarrow$ Postfix = $A \ B \ - \ D \ E \ F \ + \ G \ * \ / \ +$
- **Trace 5:** $A + B * (C - D / E \ \$ F) * G \rightarrow$ Postfix = $A \ B \ C \ D \ E \ F \ \$ \ / \ - \ * \ G \ * \ +$

Q14. Evaluation of Postfix and Prefix Expressions with Stack Traces.**[3, 4 & 7 Marks • CO2]**

Algorithm: Scan postfix left-to-right. Push operands onto stack. For operator θ , pop $OP2$, pop $OP1$, compute $VAL = OP1 \ \theta \ OP2$, and push VAL . Final answer is popped from stack.

Tabular Trace for: $10 \ 2 \ + \ 8 \ 4 \ / \ * \ 6 \ 3 \ - \ /$

Token	Operation / Stack Action	Stack Content
10, 2	Push 10, Push 2	[10, 2]
+	Pop 2, 10 $\rightarrow$ Compute $10 + 2 = 12 \rightarrow$ Push 12	[12]
8, 4	Push 8, Push 4	[12, 8, 4]
/	Pop 4, 8 $\rightarrow$ Compute $8 / 4 = 2 \rightarrow$ Push 2	[12, 2]
*	Pop 2, 12 $\rightarrow$ Compute $12 \times 2 = 24 \rightarrow$ Push 24	[24]
6, 3	Push 6, Push 3	[24, 6, 3]
-	Pop 3, 6 $\rightarrow$ Compute $6 - 3 = 3 \rightarrow$ Push 3	[24, 3]
/	Pop 3, 24 $\rightarrow$ Compute $24 / 3 = 8 \rightarrow$ Push 8	[8] $\rightarrow$ Result = 8

• **Postfix 2:** $5 \ 4 \ 6 \ + \ * \ 4 \ 9 \ 3 \ / \ + \ * \rightarrow (4+6=10) \rightarrow (5 \wedge 10=50) \rightarrow (9/3=3) \rightarrow (4+3=7) \rightarrow (50 \wedge 7) = \mathbf{350}$.

• **Prefix:** $+ \ * \ A \ B \ - \ C \ + \ C \ * \ B$ with $A=4, B=8, C=12 \rightarrow$ Scan right-to-left: $B \wedge A = 32 \rightarrow C + 32 = 44 \rightarrow C - 44 = -20 \rightarrow A \wedge B = 32 \rightarrow 32 + (-20) = \mathbf{12}$.

Q15. Explain Tower of Hanoi Problem and its Recursive Solution.**[3, 4 & 7 Marks • CO2]**

Rules: Move n disks from Source (A) to Destination (C) using Aux (B) such that: (1) Only one disk is moved at a time; (2) A larger disk never sits on a smaller disk. Minimal moves = $2^n - 1$. For $n=3$: $2^3 - 1 = \mathbf{7 \text{ moves}}$.

7 Steps for 3 Disks: (1) $A \rightarrow C$, (2) $A \rightarrow B$, (3) $C \rightarrow B$, (4) $A \rightarrow C$, (5) $B \rightarrow A$, (6) $B \rightarrow C$, (7) $A \rightarrow C$.

```
void TOH(int n, char src, char dest, char aux) { if (n == 1) { printf("Move disk 1 from %c to %c\n", src, dest); return; } TOH(n - 1, src, aux, dest); printf("Move disk %d from %c to %c\n", n, src, dest); TOH(n - 1, aux, dest, src); }
```

Q16. Write a C Program to Reverse a String Using Stack.**[7 Marks • CO5]**

```
#include <stdio.h> #include <string.h> #define MAX 100 char stack[MAX]; int top = -1; void push(char c) { if (top < MAX-1) stack[++top] = c; } char pop() { return (top >= 0) ? stack[top--] : '\0'; } int main() { char str[MAX]; int i; printf("Enter string: "); gets(str); for(i = 0; str[i] != '\0'; i++) push(str[i]); for(i = 0; top != -1; i++) str[i] = pop(); str[i] = '\0'; printf("Reversed string: %s\n", str); return 0; }
```

Q17. Explain Simple Queue Limitations and Circular Queue Algorithms.**[4 & 7 Marks • CO2]**

- **Simple Queue Limitation:** Suffers from *False Overflow*. As items are dequeued, FRONT shifts right. When REAR == N, no new element can be inserted even if initial slots are empty, wasting memory space.
- **Circular Queue:** Reconnects the last slot back to the first using modulo arithmetic ($R = (R \bmod N) + 1$), eliminating false overflow.

```

Procedure CQINSERT(F, R, Q, N, Y): 1. IF (R == N) THEN R = 1 ELSE R = R + 1 2. IF (F == R) THEN
Write('Overflow'); Return 3. Q[R] = Y 4. IF (F == 0) THEN F = 1 5. Return Procedure CQDELETE(F, R, Q, N): 1. IF (F
== 0) THEN Write('Underflow'); Return(0) 2. Y = Q[F] 3. IF (F == R) THEN F = 0; R = 0 ELSE IF (F == N) THEN F = 1
ELSE F = F + 1 4. Return(Y)

```

Q18. Explain Deque (Double Ended Queue) and Priority Queue.**[3, 4 & 7 Marks • CO2]**

- **Deque:** A linear list where insertions and deletions can occur at either end (LEFT/RIGHT). Types: (1) *Input-Restricted*: Insertion at one end only, deletions at both ends; (2) *Output-Restricted*: Deletion at one end only, insertions at both ends.
- **Priority Queue:** Elements are assigned a priority. High-priority elements are serviced before low-priority ones. Same-priority elements follow FIFO. Used extensively in operating system CPU process scheduling.

Q19. Master Comparison: Stack vs. Simple Queue vs. Priority Queue vs. Deque.**[3 & 4 Marks • CO2]**

Structure	Discipline	Pointers Used	Insertion End	Deletion End
Stack	LIFO	1 pointer (TOP)	TOP only	TOP only
Simple Queue	FIFO	2 pointers (FRONT, REAR)	REAR only	FRONT only
Circular Queue	FIFO (Circular)	2 pointers (F, R wrapped)	REAR (wrapped)	FRONT (wrapped)
Priority Queue	Priority + FIFO	Pointer to sorted head	Sorted order position	Highest priority item
Deque	Flexible	2 pointers (LEFT, RIGHT)	Both ends (or 1 end)	Both ends (or 1 end)

Q20. Singly Linked List: Operations, Algorithms, and Node Availability Stack.**[4 & 7 Marks • CO2]**

Availability Stack (AVAIL): A pool of free nodes managed for dynamic linked allocation. Free nodes are popped from AVAIL on insert and pushed back onto AVAIL on delete.

```

INSERT_FIRST(X, FIRST): 1. IF AVAIL == NULL THEN Write('Overflow'); Return(FIRST) 2. NEW = AVAIL; AVAIL =
LINK(AVAIL); INFO(NEW) = X; LINK(NEW) = FIRST; Return(NEW) INSERT_LAST(X, FIRST): 1. Allocate NEW with INFO=X,
LINK=NULL. IF FIRST == NULL THEN Return(NEW) 2. SAVE = FIRST; WHILE LINK(SAVE) != NULL DO SAVE = LINK(SAVE) 3.
LINK(SAVE) = NEW; Return(FIRST) DELETE_NODE(X, FIRST): 1. IF FIRST == NULL THEN Write('Underflow'); Return 2. Find
node X and its predecessor PRED. 3. IF X == FIRST THEN FIRST = LINK(FIRST) ELSE LINK(PRED) = LINK(X) 4. LINK(X) =
AVAIL; AVAIL = X; Return

```

Q21. Explain Doubly Linked List and Circular Linked List Operations.**[4 & 7 Marks • CO2]**

- **Doubly Linked List:** Each node has LPTR (predecessor), INFO (data), and RPTR (successor). Enables bidirectional traversal. Left insertion at M: LPTR(NEW)=LPTR(M); RPTR(NEW)=M; LPTR(M)=NEW; RPTR(LPTR(NEW))=NEW; Deletion of node OLD: RPTR(LPTR(OLD))=RPTR(OLD); LPTR(RPTR(OLD))=LPTR(OLD); Free(OLD);
- **Circular Linked List:** Last node's pointer links back to FIRST instead of NULL. Advantage: Traversal can begin from any node and reach every other node in the ring.

Q22. Linked Representation of Stack and Queue.**[4 & 7 Marks • CO2]**

- **Linked Stack:** TOP points to list head. *Push*: NEW->next = TOP; TOP = NEW; (\$O(1)\$). *Pop*: TEMP = TOP; TOP = TOP->next; free(TEMP); (\$O(1)\$). No overflow except physical memory exhaustion.
- **Linked Queue:** Maintained via FRONT and REAR pointers. *Enqueue*: Append at REAR, advance REAR (\$O(1)\$). *Dequeue*: Remove from FRONT, advance FRONT (\$O(1)\$).

Q23. Define Tree Terminology and Types of Binary Trees.**[3 & 4 Marks • C03]**

Terminology: (1) *Root*: Topmost node with in-degree 0; (2) *Leaf (Terminal)*: Node with out-degree 0; (3) *Degree*: Number of children of a node; (4) *Height/Depth*: Maximum edges from root to a leaf; (5) *Forest*: Set of disjoint trees; (6) *Fundamental Theorem*: Any tree with N nodes contains exactly $N - 1$ edges.

Binary Tree Types:

- **Full / Strictly Binary**: Every node has either 0 or 2 children.
- **Complete Binary**: All levels full except possibly the last, where nodes are packed strictly left-to-right.
- **Perfect Binary**: All internal nodes have 2 children, all leaves are at the same level. Nodes = $2^{h+1} - 1$.
- **Skewed Binary**: Every node has only 1 child (Left-skewed or Right-skewed degenerate list).

Q24. Binary Tree Traversals: Recursive and Iterative Algorithms.**[3, 4 & 7 Marks • C03]**

- **Preorder (V-L-R)**: Visit Root → Traverse Left → Traverse Right.
- **Inorder (L-V-R)**: Traverse Left → Visit Root → Traverse Right (produces sorted order in BST).
- **Postorder (L-R-V)**: Traverse Left → Traverse Right → Visit Root.

```
Recursive Algorithms: RINORDER(T): IF T != NULL THEN { RINORDER(LPTR(T)); Write(DATA(T));
RINORDER(RPTR(T)); } RPREORDER(T): IF T != NULL THEN { Write(DATA(T)); RPREORDER(LPTR(T)); RPREORDER(RPTR(T)); }
RPOSTORDER(T): IF T != NULL THEN { RPOSTORDER(LPTR(T)); RPOSTORDER(RPTR(T)); Write(DATA(T)); } Non-Recursive
(Iterative) Preorder using Stack: 1. PUSH(Root). 2. WHILE Stack not empty DO: T = POP(); Write(DATA(T)); IF
RPTR(T) != NULL THEN PUSH(RPTR(T)); IF LPTR(T) != NULL THEN PUSH(LPTR(T));
```

Q25. Constructing Binary Trees from Traversal Sequences.**[4 & 7 Marks • C03]**

- **Case 1: Preorder**: A, B, C, E, I, F, J, D, G, H, K, L | **Inorder**: E, I, C, F, J, B, G, D, K, H, L, A

1. First preorder element is Root = **A**.
2. In Inorder, all elements E, I, C, F, J, B, G, D, K, H, L are left of A → Right subtree is empty.
3. Next in preorder is **B** (Root of left subtree). In Inorder, E, I, C, F, J are left of B; G, D, K, H, L are right of B.
4. Recursing constructs the tree matching the course diagram: B has left child C and right child D; C has left child E (with right child I) and right child F (with right child J); D has left child G and right child H (with left child K, right child L).

- **Case 2: Inorder**: D, G, B, A, H, E, I, C, F | **Postorder**: G, D, B, H, I, E, F, C, A

Root is last postorder element = **A**. Inorder splits into Left: D, G, B and Right: H, E, I, C, F. Recursion determines Left subtree has root B (left child D with right child G); Right subtree has root C (left child E with children H, I; right child F).

Q26. Explain Threaded Binary Tree, Types, and Header Nodes.**[3, 4 & 7 Marks • C03]**

Motivation: In an n -node binary tree, exactly $n+1$ link fields store NULL values. Threaded binary trees replace these NULL links with **threads** to allow stackless, non-recursive traversals.

- **Right-Threaded**: NULL right link points to the node's **inorder successor**.
- **Left-Threaded**: NULL left link points to the node's **inorder predecessor**.
- **Two-Way Threaded**: Both left and right NULL links are replaced with predecessor and successor threads.
- **Header Node**: A dummy node that maintains uniformity. Left link points to the root; right link points to itself. The leftmost leaf's left thread and rightmost leaf's right thread point back to this Header Node.

Q27. Binary Expression Trees and Conversion of General Tree to Binary Tree.**[4 & 7 Marks • C03]**

- **Binary Expression Tree**: Leaves are operands; internal nodes are operators. Evaluating via Inorder yields infix; Preorder yields prefix; Postorder yields postfix. Expression $A + B * (C + D)$ has '+' at root, left child A, right child '*' (left B, right '+' with children C, D).
- **General Tree to Binary Tree Rules**: (1) Root of General Tree becomes Root of Binary Tree; (2) Left child of a node in General Tree becomes its **Left Child**; (3) Immediate right sibling in General Tree becomes its **Right Child**.

Q28. Binary Search Tree (BST): Concept, Search, and Insertion Algorithms.**[3, 4 & 7 Marks • CO3]**

BST Property: For every node X : $\text{Keys in Left Subtree} < \text{Key}(X) < \text{Keys in Right Subtree}$. Inorder traversal yields sorted order.

```
SearchElement(Tree, Val): IF Tree == NULL OR Info(Tree) == Val THEN Return Tree ELSE IF Val < Info(Tree)
THEN Return SearchElement(LPTR(Tree), Val) ELSE Return SearchElement(RPTR(Tree), Val)
InsertBST(Root, NEW): 1. IF Root == NULL THEN Root = NEW; Return 2. Traverse: IF Info(NEW) < Info(currNode) move Left ELSE move Right. 3.
Attach NEW as left or right child at the first encountered NULL leaf position.
```

Q29. BST Deletion: Explain the Three Cases and Algorithm.**[4 & 7 Marks • CO3]**

- **Case 1 (Leaf Node):** Simply set the parent's pointer to NULL and free the node.
- **Case 2 (Single Child):** Bypass the node by linking the parent pointer directly to the single child.
- **Case 3 (Two Children):** Find the node's **inorder successor** (smallest node in the right subtree) or inorder predecessor. Copy its value to the target node, then recursively delete the inorder successor (which is guaranteed to have at most one child).

Example: Deleting 70 from BST [50, 30, 70, 20, 40, 60, 80]: 70 has two children (60, 80). Inorder successor is 80. Replace 70 with 80, then prune leaf 80.

Q30. Explain AVL Trees, Balance Factor, and the Four AVL Rotations.**[3, 4 & 7 Marks • CO3]**

Definition: An AVL tree (Adelson-Velsky & Landis) is a self-balancing BST where the **Balance Factor (BF)** of every node satisfies $BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right}) \in \{-1, 0, 1\}$. Guaranteed worst-case complexity: $O(\log n)$.

The 4 Balancing Rotations:

Rotation	Cause / Insertion Position	Balancing Action Taken
LL Rotation	Inserted into Left child's Left subtree ($BF = +2$).	Single Clockwise rotation at unbalanced node A.
RR Rotation	Inserted into Right child's Right subtree ($BF = -2$).	Single Counter-Clockwise rotation at unbalanced node A.
LR Rotation	Inserted into Left child's Right subtree ($BF = +2$).	Double rotation: RR rotation on left child, then LL on node A.
RL Rotation	Inserted into Right child's Left subtree ($BF = -2$).	Double rotation: LL rotation on right child, then RR on node A.

Example: Inserting numbers 10, 20, 30 creates a right-skewed tree ($BF(10) = -2$). Applying an RR rotation pulls 20 to root with 10 (left) and 30 (right), restoring balance.

Q31. Explain B-Trees of Order m and 2-3 Trees.**[3, 4 & 7 Marks • CO3]**

- **B-Tree of Order m :** A self-balancing multi-way search tree designed for disk/block storage. Properties: (1) Each node holds at most $m-1$ keys and m children; (2) Root has at least 2 children; (3) Non-root internal nodes have at least $\lceil m/2 \rceil$ children; (4) All leaf nodes reside at the exact same depth; (5) Keys inside each node are sorted ascending.
- **2-3 Tree:** A specialized B-tree of order 3. Each node contains either 1 key (2 children: 2-node) or 2 keys (3 children: 3-node). When an insertion pushes 3 keys into a node, it splits: the middle key is promoted to the parent, and the left and right keys become two sibling 2-nodes.

Q32. Graph Definitions, Adjacency Matrix vs. Adjacency List.

[3, 4 & 7 Marks • C03]

Definitions: Complete Graph has all possible edges ($N(N-1)/2$ edges in undirected graph); Multigraph allows parallel edges; Directed Acyclic Graph (DAG) has directed edges without cycles.

Criteria	Adjacency Matrix	Adjacency List
Structure	2D boolean/weight matrix of size $V \times V$.	Array of V linked lists storing adjacent vertices.
Space Complexity	$O(V^2)$ (optimal for dense graphs).	$O(V + E)$ (optimal for sparse graphs).
Edge Lookup / Neighbors	$O(1)$ edge check; $O(V)$ to find all neighbors.	$O(\text{degree}(u))$ for lookup and neighbor traversal.

Q33. Graph Traversals: Breadth-First Search (BFS) vs. Depth-First Search (DFS).

[4 & 7 Marks • C03]

- **BFS:** Uses a **Queue**. Explores all neighbors of the current vertex before moving deeper. Guarantees shortest path in unweighted graphs. Time: $O(V + E)$, Space: $O(V)$.
- **DFS:** Uses a **Stack / Recursion**. Traverses deeply along each branch to leaf vertices before backtracking. Ideal for topological sort and cycle detection. Time: $O(V + E)$, Space: $O(V)$.

Q34. Minimum Spanning Tree (MST): Kruskal's vs. Prim's Algorithms (Solved Numerical).

[4 & 7 Marks • C03]

Definition: An acyclic subgraph of a connected, weighted graph connecting all V vertices with exactly $V-1$ edges and minimal total edge cost.

- **Kruskal's Algorithm:** Sort all edges ascending by weight. Greedily add the lowest-weight edge provided it does not form a cycle (checked via Disjoint Set Union) until $V-1$ edges are added.
- **Prim's Algorithm:** Start at arbitrary vertex. Grow MST by greedily selecting the minimum weight edge connecting a tree vertex to an unvisited fringe vertex.

Trace on Course 9-Vertex Graph (Vertices 0 to 8):

Edges selected: (7,6: w=1), (8,2: w=2), (6,5: w=2), (0,1: w=4), (2,5: w=4), (2,3: w=7), (0,7: w=8), (3,4: w=9).

Total MST Weight = $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$.

Q35. Dijkstra's Algorithm for Single-Source Shortest Path (Solved Trace A to F).

[7 Marks • C03]

Principle: Greedy algorithm to find shortest path from source to all vertices in graphs with non-negative edge weights.

Solved Trace (Graph from Notes: Find Shortest Path from A to F):

Iteration	Selected Vertex	Relaxation Steps / Distance Calculations	Distances [A, B, C, D, E, F]
Init	-	Initialize source $\text{dist}[A]=0$, all other vertices $=\infty$	[0, ∞ , ∞ , ∞ , ∞ , ∞]
1	A (0)	Relax B (cost 1), C (cost 5)	[0, 1, 5, ∞ , ∞ , ∞]
2	B (1)	Relax C: $1+1=2$; D: $1+2=3$; E: $1+4=5$	[0, 1, 2, 3, 5, ∞]
3	C (2)	No shorter edges found	[0, 1, 2, 3, 5, ∞]
4	D (3)	Relax F: $3+6=9$	[0, 1, 2, 3, 5, 9]
5	E (5)	Relax F: $5+2=7$ (shorter than 9!) $\rightarrow \text{dist}[F]$ updated to 7	[0, 1, 2, 3, 5, 7]

Shortest Path: A to B to E to F with **Minimum Cost = 7**.

Q36. What is Hashing? Explain Hash Functions and Qualities of Good Hash Function.**[3, 4 & 7 Marks • CO1]**

Concept: Hashing maps a large search key space into a smaller range of integer table indices via a mathematical function $h(k)$, enabling average $O(1)$ retrieval without sequential searches.

Qualities of a Good Hash Function: (1) Uniform distribution across table slots; (2) Minimizes collisions; (3) Extremely fast computation.

Standard Hash Functions:

1. **Division Method:** $h(k) = k \bmod M$, where M is a prime number close to table size.
2. **Multiplication Method:** $h(k) = \lfloor \text{floor } M \times (k \cdot A \bmod 1) \rfloor$ (Knuth recommends $A \approx 0.618033$).
3. **Mid-Square Method:** Square key k^2 and extract the middle digits.
4. **Folding Method:** Partition key into chunks of equal length, add them together, and discard the final carry.

Q37. Explain Collision Resolution Techniques: Open Addressing vs. Chaining.**[4 & 7 Marks • CO1]**

Collision: Occurs when two distinct keys produce the same table index ($h(k_1) = h(k_2)$).

• **Open Addressing:** All records are stored directly within the hash table array.

- **Linear Probing:** $h(k, i) = (h'(k) + i) \bmod M$. Searches next slot sequentially. Suffers from *primary clustering*.

- **Quadratic Probing:** $h(k, i) = (h'(k) + i^2) \bmod M$. Eliminates primary clustering but can cause secondary clustering.

- **Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod M$. Step size depends on a second hash function $h_2(k)$.

• **Separate Chaining:** Each table slot points to the head of a linked list containing all keys mapped to that index. Handles overflow gracefully without resizing constraints.

Q38. Solved Hashing Numerical Problems (Linear Probing, Quadratic Probing, Chaining).**[4 & 7 Marks • CO1]**

• **Problem 1: Linear Probing** on Table Size $M=10$, $h(k) = k \bmod 10$. Keys: 72, 27, 36, 24, 63, 81, 92, 101.

72 → 2, 27 → 7, 36 → 6, 24 → 4, 63 → 3, 81 → 1. Next, 92 hashes to 2 (collision) → probes 3, 4 occupied → stored at 5. Next, 101 hashes to 1 (collision) → probes 2, 3, 4, 5, 6, 7 occupied → stored at 8.

Final Table: [0:-, 1:81, 2:72, 3:63, 4:24, 5:92, 6:36, 7:27, 8:101, 9:-].

• **Problem 2: Chaining** with $M=9$, $h(k) = k \bmod 9$. Keys: 7, 24, 18, 52, 36, 54, 11, 23.

Slot 0 → 18 → 54 → 36 → NULL; Slot 2 → 11 → NULL; Slot 5 → 23 → NULL; Slot 6 → 24 → 52 → NULL; Slot 7 → 7 → NULL.

• **Rehashing:** When the table exceeds its load factor threshold (or table is full), a new table of double size (preferably a prime number) is created, and all elements are re-hashed.

Q39. File Structures & Organizations: Sequential, Indexed Sequential, and Direct/Hashed.**[3, 4 & 7 Marks • CO1]**

• **Sequential Files:** Records stored physically sorted on an ordering key. Random access requires binary search over blocks. Inserting or deleting records requires expensive shifting of physical records.

• **Indexed Sequential Files:** Primary data file is ordered, accompanied by a small, sorted Index File containing [Key, Block Pointer]. Random access searches index via binary search, then fetches target block in 1 access.

• **Direct / Hashed Files:** Records are partitioned into disk buckets using a hash function. Bucket directory resolves bucket numbers to block addresses in ~2 block accesses.

Block Access Comparison Problem (from Notes): Sequential file with $r = 1024$ records of 128 bytes on 2048-byte disk blocks: Data blocks = $(1024 \times 128) / 2048 = 64$ blocks → Binary search requires $\log_2 64 = 6$ block accesses. With an Index file (key=4B, pointer=4B → 8B/entry): Index blocks = $(1024 \times 8) / 2048 = 4$ blocks → Index search requires $\log_2 4 = 2$ accesses + 1 data block fetch = 3 total accesses (50% faster).

Q40. Internal vs. External Sorting; Bubble, Selection, and Insertion Sort.**[3, 4 & 7 Marks • CO4, CO5]**

- **Internal vs. External:** Internal sorting occurs entirely within RAM for small datasets (Bubble, Quick, Heap). External sorting handles datasets too massive for RAM by sorting disk blocks and merging runs (Merge Sort).
- **Bubble Sort:** Swaps adjacent out-of-order pairs. Optimization: A boolean swapped flag terminates execution early if no swaps occur during a pass (Best: $O(n)$, Worst: $O(n^2)$, Space: $O(1)$, Stable).
- **Selection Sort:** Repeatedly finds minimum unsorted element and swaps with current index ($O(n^2)$ all cases, Space: $O(1)$, Unstable).
- **Insertion Sort:** Inserts key into sorted prefix by shifting larger elements right (Best: $O(n)$, Worst: $O(n^2)$, Space: $O(1)$, Stable).

Q41. Quick Sort: Algorithm and Justification of Pivot Selection Impact.**[3, 4 & 7 Marks • CO4, CO5]****Does Pivot Selection affect Time Complexity? Justification:**

Yes, critically. (1) *Balanced Partitions:* When the pivot divides the array into two roughly equal halves ($n/2$), recursion depth is $\log_2 n$, resulting in Best & Average Case time of $O(n \log n)$.

(2) *Unbalanced Partitions:* If the pivot is consistently chosen as the smallest or largest element (e.g., picking the first element on an already sorted or reverse-sorted array), one partition contains 0 elements and the other $n-1$ elements. Recursion depth degrades to n , deteriorating Quick Sort into $O(n^2)$ (Selection Sort-like behavior).

Mitigation: Median-of-three or randomized pivot selection ensures $O(n \log n)$ with high probability.

Q42. Merge Sort (Complexity Derivation) and Heap Sort (Max-Heap & Heapify).**[4 & 7 Marks • CO4, CO5]**

- **Merge Sort:** Divide-and-conquer. Recurrence: $T(n) = 2T(n/2) + \Theta(n)$. By Master Theorem ($a=2, b=2, d=1 \implies a = b^d$), $T(n) = \Theta(n \log n)$ in Best, Average, and Worst cases. Requires $O(n)$ auxiliary buffer.
- **Heap Sort:** Utilizes a complete binary tree where parent $\geq$ children (Max-Heap). (1) Build_Max_Heap in $O(n)$; (2) Swap root (maximum) with last leaf, reduce heap size, and call Heapify(0) down. Repeat until sorted. Guaranteed $O(n \log n)$ time in-place with $O(1)$ auxiliary memory.

Q43. Master Summary Comparison Table of All Sorting Algorithms.**[4 & 7 Marks • CO4, CO5]**

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stability
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$ (In-place)	Stable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$ (In-place)	Unstable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$ (In-place)	Stable
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ stack	Unstable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ auxiliary	Stable
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ (In-place)	Unstable

Q44. Linear Search vs. Binary Search (Comparison & Solved Search Trace).**[3, 4 & 7 Marks • CO4, CO5]**

- **Linear Search:** Sequentially compares elements. Works on unsorted arrays. Time: Best $O(1)$, Worst $O(n)$. Space: $O(1)$.
- **Binary Search:** Divide-and-conquer on **sorted arrays**. Computes $MID = \lfloor (BEG + END)/2 \rfloor$. Halves search interval each comparison. Time: Best $O(1)$, Worst $O(\log n)$. Space: $O(1)$.

Solved Trace (Search 50 in [10, 14, 20, 39, 41, 45, 49, 50, 60]):

- Pass 1: $BEG = 0, END = 8 \implies MID = 4$ ($A[4] = 41$). Since $41 < 50 \implies BEG = MID + 1 = 5$.
- Pass 2: $BEG = 5, END = 8 \implies MID = 6$ ($A[6] = 49$). Since $49 < 50 \implies BEG = MID + 1 = 7$.
- Pass 3: $BEG = 7, END = 8 \implies MID = 7$ ($A[7] = 50$). Element matched at index **7** (location 8).